Machine Learning Methods for Finance

Unsupervised Learning - Generative Models

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Intro to Generative Models

Generative aka Simulation

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance
- In the following we will focus on perhaps the most common, and certainly the most relevant to our use case

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance
- In the following we will focus on perhaps the most common, and certainly the most relevant to our use case
- Namely, we will simulate asset returns

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance
- In the following we will focus on perhaps the most common, and certainly the most relevant to our use case
- Namely, we will simulate asset returns
- Note that this exercise is cast in a time series context; i.e. we will simulate the evolution of returns over time

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance
- In the following we will focus on perhaps the most common, and certainly the most relevant to our use case
- Namely, we will simulate asset returns
- Note that this exercise is cast in a time series context; i.e. we will simulate the evolution of returns over time
- However, the simulation methods here are i) not built to capture temporal persistence (see our Time Series notes) and ii) can be applied in myriad cross sectional settings.

Modeling Returns

Q: Why usually simulate returns, not prices?

Modeling Returns

Q: Why usually simulate returns, not prices?

Defining Returns

Q: Simple vs Log? Definition and Use Case?

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  -
- Log Returns are computed as

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  -
- Log Returns are computed as $r_t = ln(P_t/P_{t-1}) = ln(1 + R_t)$

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  -
- Log Returns are computed as $r_t = ln(P_t/P_{t-1}) = ln(1 + R_t)$
  -

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  -
- Log Returns are computed as $r_t = ln(P_t/P_{t-1}) = ln(1 + R_t)$
  -

Q: Net vs Gross? Definition and Use Case

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  -
- Log Returns are computed as $r_t = ln(P_t/P_{t-1}) = ln(1 + R_t)$
  -

Q: Net vs Gross? Definition and Use Case
A:

-
-
-
-

Simulating with Simple Returns is Difficult

Q: Problems with simulating Gaussian Simple Returns?

Two problems arise if we assume $R_t \sim iid\ N(\mu, \sigma^2)$

Simulating with Simple Returns is Difficult

Q: Problems with simulating Gaussian Simple Returns?

Two problems arise if we assume $R_t \sim iid\ N(\mu, \sigma^2)$

1.
2.

Solution:

Simulating with Simple Returns is Difficult

Q: Problems with simulating Gaussian Simple Returns?

Two problems arise if we assume $R_t \sim iid\ N(\mu, \sigma^2)$

1.
2.

Solution:

-
-

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

$$\exp(r_t) = \exp\big(\ln(1 + R_t)\big) = 1 + R_t, \quad \text{range}(\exp) = (0, \infty)$$

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

$$\exp(r_t) = \exp\big(\ln(1 + R_t)\big) = 1 + R_t, \quad \text{range}(\exp) = (0, \infty)$$

$$\Rightarrow \quad 1 + R_t > 0 \implies R_t > -1$$

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

$$\exp(r_t) = \exp\big(\ln(1 + R_t)\big) = 1 + R_t, \quad \text{range}(\exp) = (0, \infty)$$

$$\Rightarrow \quad 1 + R_t > 0 \implies R_t > -1$$

$$\Rightarrow \quad \frac{P_t}{P_{t-1}} - 1 > -1$$

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

$$\exp(r_t) = \exp\big(\ln(1 + R_t)\big) = 1 + R_t, \quad \text{range}(\exp) = (0, \infty)$$

$$\Rightarrow \quad 1 + R_t > 0 \implies R_t > -1$$

$$\Rightarrow \quad \frac{P_t}{P_{t-1}} - 1 > -1$$

$$\Rightarrow \quad \frac{P_t}{P_{t-1}} > 0 \implies P_t > 0 \quad (\text{if } P_{t-1} > 0).$$

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

$$1 + R_t(H) \;=\; \prod_{j=0}^{H-1} \big(1 + R_{t-j}\big).$$

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

$$1 + R_t(H) \;=\; \prod_{j=0}^{H-1} \big(1 + R_{t-j}\big).$$

Using $1 + R_{t-j} = \exp(r_{t-j})$, we obtain

$$1 + R_t(H) = \prod_{j=0}^{H-1} \exp\left(r_{t-j}\right) = \exp\Big(\sum_{j=0}^{H-1} r_{t-j}\Big).$$

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

$$1 + R_t(H) \;=\; \prod_{j=0}^{H-1} \big(1 + R_{t-j}\big).$$

Using $1 + R_{t-j} = \exp(r_{t-j})$, we obtain

$$1 + R_t(H) = \prod_{j=0}^{H-1} \exp\left(r_{t-j}\right) = \exp\Big(\sum_{j=0}^{H-1} r_{t-j}\Big).$$

Therefore, the multi-period log return is

$$r_t(H) = \ln\left(1 + R_t(H)\right) = \sum_{j=0}^{H-1} r_{t-j}.$$

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

$$1 + R_t(H) = \prod_{j=0}^{H-1} \big(1 + R_{t-j}\big).$$

Using $1 + R_{t-j} = \exp(r_{t-j})$, we obtain

$$1 + R_t(H) = \prod_{j=0}^{H-1} \exp\left(r_{t-j}\right) = \exp\Big(\sum_{j=0}^{H-1} r_{t-j}\Big).$$

Therefore, the multi-period log return is

$$r_t(H) = \ln\big(1 + R_t(H)\big) = \sum_{j=0}^{H-1} r_{t-j}.$$

Note: The sum of Gaussian random variables is Gaussian. Hence, if each r_{t-j} is Gaussian (under appropriate independence or dependence conditions), then $r_t(H)$ is Gaussian.

Building a Momentum Signal via Simulation

Q: How might we use simulation to build a momentum based signal?

Building a Momentum Signal via Simulation

Q: How might we use simulation to build a momentum based signal?

1. Compute the most recent momentum value for asset A

Building a Momentum Signal via Simulation

Q: How might we use simulation to build a momentum based signal?

1. Compute the most recent momentum value for asset A
2.
3.
4.
5.
6.

Building a Momentum Signal via Simulation

Q: How might we use simulation to build a momentum based signal?

1. Compute the most recent momentum value for asset A
2.
3.
4.
5.
6.

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Parametric Simulation

- Classical parametric simulation assumes a Data Generating Process (DGP) for returns

Parametric Simulation

- Classical parametric simulation assumes a Data Generating Process (DGP) for returns
- A DGP typically has two components:

$$r_t = \underbrace{\mu_t}_{\text{systematic component}} + \underbrace{\epsilon_t}_{\text{innovation (shock)}}$$

Parametric Simulation

- Classical parametric simulation assumes a Data Generating Process (DGP) for returns
- A DGP typically has two components:

$$r_t = \underbrace{\mu_t}_{\text{systematic component}} + \underbrace{\epsilon_t}_{\text{innovation (shock)}}$$

- Simplest case (pure noise):

$$r_t = \epsilon_t, \qquad \epsilon_t \sim N(0, \sigma^2)$$

Parametric Simulation

- Classical parametric simulation assumes a Data Generating Process (DGP) for returns
- A DGP typically has two components:

$$r_t = \underbrace{\mu_t}_{\text{systematic component}} + \underbrace{\epsilon_t}_{\text{innovation (shock)}}$$

- Simplest case (pure noise):

$$r_t = \epsilon_t, \qquad \epsilon_t \sim N(0, \sigma^2)$$

- More structured example:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Building a Momentum Signal with Gaussian Simulation

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path
5. Estimate the mean and standard deviation of simulated momentum

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path
5. Estimate the mean and standard deviation of simulated momentum
6. Compute the z-score of the observed momentum relative to the simulated distribution

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path
5. Estimate the mean and standard deviation of simulated momentum
6. Compute the z-score of the observed momentum relative to the simulated distribution
7. Repeat for all assets in the universe

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path
5. Estimate the mean and standard deviation of simulated momentum
6. Compute the z-score of the observed momentum relative to the simulated distribution
7. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\left[p^{(+)}(X_{i,t}, Z_t)\right] - \mathbf{1}\left[p^{(-)}(X_{i,t}, Z_t)\right]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation:

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation:

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n is the number of observations, $h > 0$

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n is the number of observations, $h > 0$ is the smoothing parameter (aka bandwidth), and $K(.)$

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n is the number of observations, $h > 0$ is the smoothing parameter (aka bandwidth), and $K(.)$ is the kernel function, which integrates to 1.

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n is the number of observations, $h > 0$ is the smoothing parameter (aka bandwidth), and $K(.)$ is the kernel function, which integrates to 1.
- Many Kernels are possible, such as Gaussian, Uniform, Triangular, etc...

From Data to a Density

How KDE builds a smooth analogue of a histogram

1. Form empirical distribution

If we treat the raw data as a density:

$$\hat{f}_{\text{emp}}(x) = \frac{1}{n} \sum_{i=1}^{n} \delta(x - x_i).$$

- Each observation is a *point mass* aka "spike"
- Graphically: vertical spikes at the observed values
- This is the raw input into KDE

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"
- The bump is the *kernel*, centered at x_i

$$\delta(x - x_i) \longrightarrow \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"
- The bump is the *kernel*, centered at x_i

$$\delta(x - x_i) \longrightarrow \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

- Each observation contributes a local density "hill"

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"
- The bump is the *kernel*, centered at x_i

$$\delta(x - x_i) \longrightarrow \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

- Each observation contributes a local density "hill"
- Shape determined by the kernel K

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"
- The bump is the *kernel*, centered at x_i

$$\delta(x - x_i) \longrightarrow \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

- Each observation contributes a local density "hill"
- Shape determined by the kernel K
- Width controlled by the bandwidth h

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

Graphically:

- Place one kernel at each data point

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

Graphically:

- Place one kernel at each data point
- Scale each kernel so its total area equals $1/n$

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

Graphically:

- Place one kernel at each data point
- Scale each kernel so its total area equals $1/n$
- Add all kernels together

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

Graphically:

- Place one kernel at each data point
- Scale each kernel so its total area equals $1/n$
- Add all kernels together

The resulting density curve is:

- Smooth (if the kernel is smooth)
- Continuous (no bins)
- Higher in regions where many kernel bumps overlap

Simulating from a Kernel Density Estimate

Q: How simulate data from the KDE?

Simulating from a Kernel Density Estimate

Q: How simulate data from the KDE?

1.
2.
   -
   -
   -
3.

$$\tilde{x} = x_i + \varepsilon$$

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

1.
2.
3.
4. Estimate the mean and standard deviation of simulated momentum

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

1.
2.
3.
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

1.
2.
3.
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

1.
2.
3.
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

How we'll use Diffusion Models

- Like parametric and KDE, we will use Diffusion models to simulate returns

How we'll use Diffusion Models

- Like parametric and KDE, we will use Diffusion models to simulate returns
- Diffusion models are inherently non parametric, and in that sense are similar to KDE

How we'll use Diffusion Models

- Like parametric and KDE, we will use Diffusion models to simulate returns
- Diffusion models are inherently non parametric, and in that sense are similar to KDE
- As in both parametric and KDE methods, Diffusion models train and then simulate. However, Diffused are trained by simulating noisy versions of the data and learning to reverse that process.

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i;
  $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise
- After many steps, the data becomes indistinguishable from pure noise

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise
- After many steps, the data becomes indistinguishable from pure noise
- Train a neural network to reverse this corruption, step by step

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise
- After many steps, the data becomes indistinguishable from pure noise
- Train a neural network to reverse this corruption, step by step
- Generate new samples by starting from noise and denoising

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of return for asset i; $x_0 = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise
- After many steps, the data becomes indistinguishable from pure noise
- Train a neural network to reverse this corruption, step by step
- Generate new samples by starting from noise and denoising

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data
- Fully specified by the noise schedule $\{\alpha_l\}$

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data
- Fully specified by the noise schedule $\{\alpha_l\}$

Reverse process (learned):

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data
- Fully specified by the noise schedule $\{\alpha_l\}$

Reverse process (learned):

$$x_L \xrightarrow{p_\theta(x_{L-1}|x_L)} x_{L-1} \xrightarrow{\cdots} x_1 \xrightarrow{p_\theta(x_0|x_1)} x_0$$

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data
- Fully specified by the noise schedule $\{\alpha_l\}$

Reverse process (learned):

$$x_L \xrightarrow{p_\theta(x_{L-1}|x_L)} x_{L-1} \xrightarrow{\cdots} x_1 \xrightarrow{p_\theta(x_0|x_1)} x_0$$

- Learns to undo the noise

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys all structure in the data
- Fully specified by the noise schedule $\{\alpha_l\}$

Reverse process (learned):

$$x_L \xrightarrow{p_\theta(x_{L-1}|x_L)} x_{L-1} \xrightarrow{\cdots} x_1 \xrightarrow{p_\theta(x_0|x_1)} x_0$$

- Learns to undo the noise
- Parameterized by a neural network

Forward Diffusion Process

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- $\alpha_l \in (0, 1)$ controls the signal-to-noise ratio

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- $\alpha_l \in (0, 1)$ controls the signal-to-noise ratio
- As l increases (i.e. the number of noise levels rises) , the signal fades and noise dominates

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- $\alpha_l \in (0, 1)$ controls the signal-to-noise ratio
- As l increases (i.e. the number of noise levels rises) , the signal fades and noise dominates
- For large l, x_l is approximately Gaussian

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- $\alpha_l \in (0, 1)$ controls the signal-to-noise ratio
- As l increases (i.e. the number of noise levels rises) , the signal fades and noise dominates
- For large l, x_l is approximately Gaussian

This formulation allows sampling x_l directly from x_0 without simulating all intermediate steps.

Training Diffusion Models (DDPM)

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs train the model to predict the noise added at each step.

Training Diffusion Models (DDPM)

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs train the model to predict the noise added at each step.

Objective (score matching / noise prediction):

$$\min_\theta \; \mathbb{E}_{l,\, x_0,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_l, l)\right\|^2\right]$$

Training Diffusion Models (DDPM)

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs train the model to predict the noise added at each step.

Objective (score matching / noise prediction):

$$\min_\theta \ \mathbb{E}_{l,\, x_0,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_l, l)\right\|^2\right]$$

where:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

- $\epsilon_\theta(x_l, l)$ is a neural network

Training Diffusion Models (DDPM)

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs train the model to predict the noise added at each step.

Objective (score matching / noise prediction):

$$\min_\theta \ \mathbb{E}_{l,\, x_0,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_l, l)\right\|^2\right]$$

where:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

- $\epsilon_\theta(x_l, l)$ is a neural network
- The network learns to identify and remove noise

Training Diffusion Models (DDPM)

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs train the model to predict the noise added at each step.

Objective (score matching / noise prediction):

$$\min_\theta \ \mathbb{E}_{l,\, x_0,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_l, l)\right\|^2\right]$$

where:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

- $\epsilon_\theta(x_l, l)$ is a neural network
- The network learns to identify and remove noise
- This loss is equivalent to maximizing a variational lower bound

Generating New Samples with Diffusion

Generating New Samples with Diffusion

Sampling procedure:

- Start from pure noise: $x_L \sim \mathcal{N}(0, I)$

Generating New Samples with Diffusion

Sampling procedure:

- Start from pure noise: $x_L \sim \mathcal{N}(0, I)$
- Iteratively apply the learned reverse transitions:

$$x_{l-1} \sim p_\theta(x_{l-1} \mid x_l)$$

Generating New Samples with Diffusion

Sampling procedure:

- Start from pure noise: $x_L \sim \mathcal{N}(0, I)$
- Iteratively apply the learned reverse transitions:

$$x_{l-1} \sim p_\theta(x_{l-1} \mid x_l)$$

- Continue until reaching x_0

Generating New Samples with Diffusion

Sampling procedure:

- Start from pure noise: $x_L \sim \mathcal{N}(0, I)$
- Iteratively apply the learned reverse transitions:

$$x_{l-1} \sim p_\theta(x_{l-1} \mid x_l)$$

- Continue until reaching x_0

Each step slightly denoises the sample; many small corrections produce a realistic final draw.

What Does DDPM Mean?

What Does DDPM Mean?

DDPM = Denoising Diffusion Probabilistic Model

- **Denoising**: the model learns to remove noise

What Does DDPM Mean?

DDPM = Denoising Diffusion Probabilistic Model

- **Denoising**: the model learns to remove noise
- **Diffusion**: noise is added gradually over many steps

What Does DDPM Mean?

DDPM = Denoising Diffusion Probabilistic Model

- **Denoising**: the model learns to remove noise
- **Diffusion**: noise is added gradually over many steps
- **Probabilistic**: each step defines a conditional distribution.

Diffusion Models: Construction vs Simulation

Model Training (Learning)

Goal: Learn the reverse process that undoes noise.

Inputs

- Real data samples $x_0 \sim p_{\text{data}}$
- Known forward noising process $q(x_l \mid x_{l-1})$

What happens

- Add noise to data (forward diffusion)
- Train a neural network to predict / remove noise
- Optimize parameters θ via a loss function

Output

- A trained diffusion model $p_\theta(x_{l-1} \mid x_l)$

Simulation (Sampling)

Goal: Generate synthetic data.

Inputs

- Pure noise $x_L \sim \mathcal{N}(0, I)$
- Trained model parameters θ

What happens

- Iteratively apply the learned reverse process
- Gradually denoise the sample

Output

- Simulated sample $\tilde{x}_0 \sim p_\theta$

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

How we'll use Generative Adversarial Networks

- Like the other generative approaches of parametric, KDE, and Diffusion models, we will use GAN's to simulate returns

How we'll use Generative Adversarial Networks

- Like the other generative approaches of parametric, KDE, and Diffusion models, we will use GAN's to simulate returns
- GANs are inherently non parametric, akin to KDE and Diffusion

How we'll use Generative Adversarial Networks

- Like the other generative approaches of parametric, KDE, and Diffusion models, we will use GAN's to simulate returns
- GANs are inherently non parametric, akin to KDE and Diffusion
- GANs are trained by learning a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, ...\}$

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.
- Training is a *minimax game*: D tries to detect fakes; G tries to fool D.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.
- Training is a *minimax game*: D tries to detect fakes; G tries to fool D.
- At equilibrium, G reproduces the true data distribution and D cannot distinguish real from synthetic.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.
- Training is a *minimax game*: D tries to detect fakes; G tries to fool D.
- At equilibrium, G reproduces the true data distribution and D cannot distinguish real from synthetic.
- Unlike VAEs, GANs learn without specifying an explicit likelihood model.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.
- Training is a *minimax game*: D tries to detect fakes; G tries to fool D.
- At equilibrium, G reproduces the true data distribution and D cannot distinguish real from synthetic.
- Unlike VAEs, GANs learn without specifying an explicit likelihood model.
- Common variants stabilize training (WGAN, WGAN-GP, spectral norm).

GAN Architecture

Learn a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

GAN Architecture

Learn a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

Architecture:

$$z \sim p(z) \xrightarrow{\text{Generator } G_\theta} \tilde{x} \qquad x,\ \tilde{x} \xrightarrow{\text{Discriminator } D_\phi} \text{prob(real)}$$

GAN Architecture

Learn a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

Architecture:

$$z \sim p(z) \xrightarrow{\text{Generator } G_\theta} \tilde{x} \qquad x,\, \tilde{x} \xrightarrow{\text{Discriminator } D_\phi} \text{prob(real)}$$

Optimization Problem (Unsupervised):

$$\min_{G_\theta} \max_{D_\phi} \Big[\mathbb{E}_{x \sim p_{\text{data}}} \big[\log D_\phi(x) \big] + \mathbb{E}_{z \sim p(z)} \big[\log(1 - D_\phi(G_\theta(z))) \big] \Big]$$

GAN Mappings

Generator / Discriminator and the Role of Embeddings

GAN Mappings

Generator / Discriminator and the Role of Embeddings

Generator Mapping:
A typical generator takes noise and optional conditioning variables:

$$h = \phi(W_z z + W_c c + b_h), \qquad \tilde{x} = G_\theta(z, c) = W_g h + b_g,$$

where c may include static features or a learned embedding identifying the asset.

GAN Mappings

Generator / Discriminator and the Role of Embeddings

Generator Mapping:
A typical generator takes noise and optional conditioning variables:

$$h = \phi(W_z z + W_c c + b_h), \qquad \tilde{x} = G_\theta(z, c) = W_g h + b_g,$$

where c may include static features or a learned embedding identifying the asset.

Discriminator Mapping:

$$h_D = \phi(W_x x + W'_c c + b_D), \qquad D_\phi(x, c) = \sigma(W_d h_D + b_d),$$

which estimates $\Pr(\text{real} \mid x, c)$.

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1.
2.
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets